

Data Abstraction in Python

Data abstraction in Python is an important concept in object-oriented programming that helps simplify complex code by hiding unnecessary implementation details. It allows us to focus only on relevant parts, making programs more user-friendly, scalable, and easier to maintain.

- **Hides Irrelevant Details:** It hides sensitive or unnecessary internal logic, showing only the necessary features to the user.
- **Real-Life Example:** Just as with watching a tutorial, we see the content but not the editing or publishing process behind it.
- **Encourages Generalization:** Instead of creating a function for a specific case, we build one that works for all relevant inputs.
- **Improves Flexibility:** By abstracting logic, we reduce complexity and make the program more reusable and easier to scale.

What Is Data Abstraction in Python?

Data abstraction is the process of hiding unnecessary and complex implementation details from users while showing only essential details and functionalities. This is a core concept in object-oriented programming that allows users to implement complex logic based on the information provided without bothering with backend complexities.

We can achieve Python abstraction by using abstract classes created through the abstract base class (abc) module and its `@abstractmethod`.

Abstraction Classes in Python

An abstract class in Python is a class that contains one or more abstract methods, methods that are declared but not implemented. It acts like a blueprint for other classes.

We can't create objects directly from an abstract class. Instead, we create objects from its subclasses, which must implement the abstract methods.

Why Use Abstract Classes?

- To set a common structure for all subclasses
- They define structure but leave implementation to derived classes.
- They help maintain consistency in code.

How to Create an Abstract Class in Python

We use the abc module to define abstract classes.

Syntax:

```
from abc import ABC, abstractmethod
```

```
class MyClass(ABC):
```

```
    @abstractmethod
```

```
    def my_method(self):
```

```
        pass
```

Key Points

- We can't make objects from abstract classes directly
- They contain methods without implementation (abstract methods).
- Subclasses must override and implement those methods.
- it helps us write cleaner, more structure code.

Example:

```
In [1]: from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        return "Bark"

# Creating an object of the Dog class
dog = Dog()
print(dog.sound())
```

Bark

Using the abc Module in Python

To implement data abstraction in Python, we use the abc module to create abstract classes. These classes define methods that their subclasses must implement.

Let's consider a real-world example. We create an abstract class named Car and then derive two subclasses, Kia and Honda. Each subclass provides its own version of the methods defined in the parent abstract class.

In this example:

- Car is the abstract base class.
- Kia and Honda are concrete subclasses that implement the abstract methods.
- We can't create an object of the Car class directly.
- To access the functionality, we must instantiate the derived classes.

Example:

```
In [2]: from abc import ABC, abstractmethod

# Abstract class
class Car(ABC):
    @abstractmethod
    def start_engine(self):
        pass

    @abstractmethod
    def stop_engine(self):
        pass

# Subclass 1
class Kia(Car):
    def start_engine(self):
        return "Kia engine started"

    def stop_engine(self):
        return "Kia engine stopped"

# Subclass 2
class Honda(Car):
    def start_engine(self):
        return "Honda engine started"

    def stop_engine(self):
        return "Honda engine stopped"

# Creating objects of the derived classes
kia = Kia()
honda = Honda()

# Calling the methods
print(kia.start_engine())    # Output: Kia engine started
print(kia.stop_engine())    # Output: Kia engine stopped

print(honda.start_engine())  # Output: Honda engine started
print(honda.stop_engine())  # Output: Honda engine stopped
```

```
Kia engine started
Kia engine stopped
Honda engine started
Honda engine stopped
```

Why use an abstract base class in Python?

An abstract base class (ABC) lets us define a common interface for multiple subclasses. It helps maintain consistency, especially in large projects or teams, so we don't need to remember all the individual class implementations; we just need to remember the shared structure (API).

Working with Abstract Class in Python

While most high-level languages support abstract classes and methods directly, Python doesn't offer abstract methods by default. Instead, we use the built-in abc module to define abstract base classes.

Key Features of the abc Module:

- ABC class: Used to create abstract base classes through inheritance.
- @abstractmethod decorator: Marks methods that must be implemented in any subclass.
- Virtual Subclasses: We can register a class as a virtual subclass without inheritance, and it will still work with `isinstance()` and `issubclass()`.

Defining an Abstract Base Class

Here's how we can define an abstract base class in Python:

```
from abc import ABC, abstractmethod
```

```
class MyABC(ABC):
```

```
    @abstractmethod
    def my_method(self):
        pass
```

- MyABC is the abstract class.
- my_method() is an abstract method that must be defined in any subclass.

Creating a Subclass

```
class ConcreteClass(MyABC):
```

```
    def my_method(self):
        print("Method implemented in subclass")
```

If a subclass doesn't implement all abstract methods, Python will raise a `TypeError` when trying to create an object of that class.

If a subclass doesn't implement all abstract methods, Python will raise a `TypeError` when trying to create an object of that class.

Concrete Method in the Abstract Base Class In Python

In Python, a concrete method is a method defined inside an abstract class with a complete implementation.

- Abstract classes can have both abstract methods (must be implemented in subclasses) and concrete methods (already implemented).
- Concrete methods help avoid code repetition in subclasses when the logic is the same.
- We access concrete methods through subclass objects. However, subclasses must implement all abstract methods; otherwise, Python raises an error.

Example:

```
In [3]: from abc import ABC, abstractmethod

class MyABC(ABC):
    @abstractmethod
    def abstract_method1(self):
        """Must be implemented by subclass"""
        pass

    @abstractmethod
    def abstract_method2(self):
        """Must be implemented by subclass"""
        pass

    def concrete_method(self):
        """Shared logic across all subclasses"""
        return "This is a concrete method from abstract class"

class ConcreteClass(MyABC):
    def abstract_method1(self):
        return "Implementation of abstract_method1"

    def abstract_method2(self):
        return "Implementation of abstract_method2"

# Creating an object of the subclass
obj = ConcreteClass()

# Accessing methods
print(obj.abstract_method1())
print(obj.abstract_method2())
print(obj.concrete_method())
```

Implementation of abstract_method1

Implementation of abstract_method2

This is a concrete method from abstract class

How to Create an Abstract Base Class and an Abstract Method in Python?

In Python, we use the abc module (stands for Abstract Base Classes) to create abstract classes and abstract methods.

Steps to Create:

1. Import ABC and abstractmethod from the abc module.
2. Inherit your class from ABC.
3. Use the @abstractmethod decorator for any method you want to make abstract.
4. Subclasses must implement all abstract methods, or Python will raise a TypeError.

Example:

```
In [4]: from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

# Creating an object of subclass
c = Circle(5)
print(c.area())
```

78.5

```
In [5]: from abc import ABC, abstractmethod

# Define an abstract base class
class Shape(ABC):
    @abstractmethod
    def area(self):
        """Calculates and returns the area of the shape."""
        pass

    @abstractmethod
    def perimeter(self):
        """Calculates and returns the perimeter of the shape."""
        pass

    def description(self):
        """Provides a general description of a shape."""
        return "This is a geometric shape."

# Define a concrete subclass: Rectangle
class Rectangle(Shape):
```

```

def __init__(self, width, height):
    self.width = width
    self.height = height

def area(self):
    return self.width * self.height

def perimeter(self):
    return 2 * (self.width + self.height)

# Define another concrete subclass: Circle
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius**2

    def perimeter(self):
        return 2 * 3.14159 * self.radius

# Attempting to instantiate the abstract class will raise an error
# try:
#     abstract_shape = Shape()
# except TypeError as e:
#     print(f"Error: {e}")

# Instantiate concrete subclasses
rectangle = Rectangle(5, 10)
circle = Circle(7)

# Call methods on the concrete objects
print(f"Rectangle Area: {rectangle.area()}")
print(f"Rectangle Perimeter: {rectangle.perimeter()}")
print(f"Rectangle Description: {rectangle.description()}")

print(f"Circle Area: {circle.area()}")
print(f"Circle Perimeter: {circle.perimeter()}")
print(f"Circle Description: {circle.description()}")

```

```

Rectangle Area: 50
Rectangle Perimeter: 30
Rectangle Description: This is a geometric shape.
Circle Area: 153.93791
Circle Perimeter: 43.98226
Circle Description: This is a geometric shape.

```